

Actividad Integradora Funcional Programación III

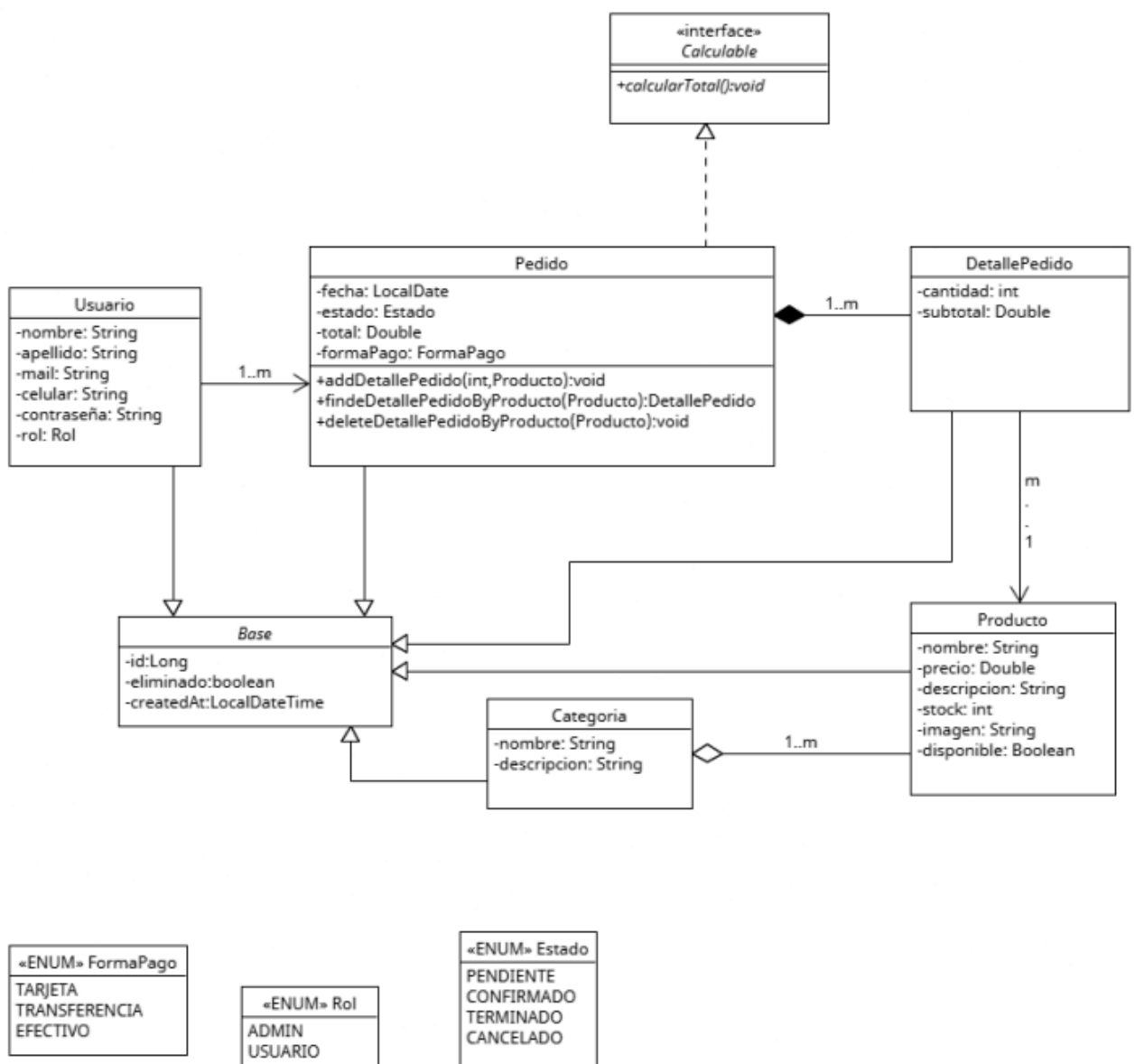
Coordinador: Enferrel, Ariel
Profesor/es: Rigoni, Cinthia
Alumno/a: Lauk, Karen

Objetivo

Practicar operaciones intermedias y terminales de los Streams en Java para procesar colecciones de forma declarativa

Caso Práctico

UML



Utilizando programación funcional.

- 1) Desarrolle un método en clase Pedido que se encargue de calcular el total.

Archivo Pedido

```
// Calcula el total del pedido
@Override 2 usages
public void calcularTotal() {
    total = detalles.stream() Stream<DetallePedido>
        .mapToDouble(DetallePedido::getSubtotal) DoubleStream
        .sum();
}
```

Main

```
System.out.println("1- TOTALES CALCULADOS");
System.out.println("    Pedido #101 Total: $" + pedido1.getTotal());
System.out.println("    Pedido #102 Total: $" + pedido2.getTotal());
System.out.println("-----");
```

Resultado

```
-----
1- TOTALES CALCULADOS
    Pedido #101 Total: $53000.0
    Pedido #102 Total: $18500.0
-----
```

Se implementó el método `calcularTotal()` dentro de la clase `Pedido`. Para resolver esta consigna se utilizó programación funcional mediante Streams de Java. El método recorre la colección de detalles del pedido, obtiene cada subtotal mediante `mapToDouble()` y finalmente calcula la suma total utilizando `sum()`.

De esta forma se obtiene el importe total del pedido sin necesidad de utilizar estructuras iterativas tradicionales como `for` o `while`.

- 2) Mostrar por consola productos disponibles

Main

```
System.out.println("2- PRODUCTOS DISPONIBLES:");
menu.stream()
    .filter(Producto::getDisponible)
    .forEach( Producto producto ->
        System.out.println(producto.getNombre())
    );
System.out.println("-----");
```

Resultado

```
2- PRODUCTOS DISPONIBLES:  
Bebida Cola  
Pizza Mozzarella  
Hamburguesa Triple  
Hamburguesa Smash Especial  
-----
```

Para mostrar únicamente los productos disponibles se utilizó la operación intermedia `filter()`, que permite seleccionar los elementos que cumplen una determinada condición. La operación terminal `forEach()` se utilizó para imprimir por consola los productos filtrados.

- 3) Mostrar por consola la cantidad de ítems que tiene un pedido (ejemplo 2 panchos, 2 bebidas, deberá de decir que hay 4 ítems)

Main

```
System.out.println("3- UNIDADES TOTALES POR PEDIDO:");  
int itemsP1 = pedido1.getDetalles().stream().mapToInt(DetallePedido::getCantidad).sum();  
System.out.println("    Pedido #101 tiene: " + itemsP1 + " ítems en total.");  
System.out.println("-----");
```

Resultado

```
3- UNIDADES TOTALES POR PEDIDO:  
    Pedido #101 tiene: 4 ítems en total.  
-----
```

Se utilizó un `Stream` sobre la colección de detalles del pedido. Mediante `mapToInt()` se obtuvo la cantidad de unidades de cada producto y luego se sumaron utilizando `sum()`. Por ejemplo, si un pedido contiene 2 hamburguesas y 2 bebidas, el resultado será 4 ítems en total.

- 4) Detectar productos que tengan menos de 5 como valor en stock

Main

```
System.out.println("4- PRODUCTOS CON STOCK CRÍTICO:");  
menu.stream()  
    .filter( Producto producto -> producto.getStock() < 5)  
    .forEach( Producto producto ->  
        System.out.println(producto.getNombre()  
            + " - Stock: "  
            + producto.getStock())  
    );  
System.out.println("-----");
```

Resultado

```
-----  
4- PRODUCTOS CON STOCK CRÍTICO:  
Cerveza Tirada - Stock: 0  
Hamburguesa Smash Especial - Stock: 3  
-----
```

Se utilizó la operación `filter()` para identificar aquellos productos cuyo stock es inferior a 5 unidades.

De esta manera se pueden detectar fácilmente los productos con stock crítico y mostrarlos por consola para su control.

Conclusión

El trabajo permitió aplicar programación funcional utilizando Streams de Java, empleando operaciones como `filter()`, `mapToInt()`, `mapToDouble()` y `sum()` para resolver distintos problemas de procesamiento de colecciones de manera declarativa y eficiente.

Resultado final

```
Practico del mod 7 - Programacion Funcional
```

```
-----  
1- TOTALES CALCULADOS
```

```
    Pedido #101 Total: $53000.0
```

```
    Pedido #102 Total: $18500.0  
-----
```

```
2- PRODUCTOS DISPONIBLES:
```

```
Bebida Cola
```

```
Pizza Mozzarella
```

```
Hamburguesa Triple
```

```
Hamburguesa Smash Especial  
-----
```

```
3- UNIDADES TOTALES POR PEDIDO:
```

```
    Pedido #101 tiene: 4 items en total.  
-----
```

```
4- PRODUCTOS CON STOCK CRÍTICO:
```

```
Cerveza Tirada - Stock: 0
```

```
Hamburguesa Smash Especial - Stock: 3  
-----
```